langchain

보완, 기능 많.

12장

노드

엣지

상태

checkpoint

AI
APP

개발할 때 사용하는
라이브러리

언제?
무엇을?

랭그래프와 친해지기

langchain에서는 어렵..

랭그래프는 언어 모델을 활용하여 더 복잡한 일을 하도록 설계할 수 있는 프레임워크입니다. 랭그래프를 이용하면 하나의 AI 에이전트가 일을 수행하는 방식을 구체화할 수 있고, 각각 전문 역량을 갖춘 여러 AI 에이전트가 서로 협력하도록 만들 수도 있습니다. 이번 장에서는 랭그래프를 이용하여 단순한 작업부터 글쓰기와 같은 복잡한 업무까지 자동으로 수행하는 프로그램을 구현해 보겠습니다.

12-1 랭그래프로 만드는 기본 챗봇
12-2 대화 내용을 저장하는 메모리
12-3 인터넷 검색 후 기사를 작성하는 챗봇 만들기

12-1 랭그래프로 만드는 기본 챗봇

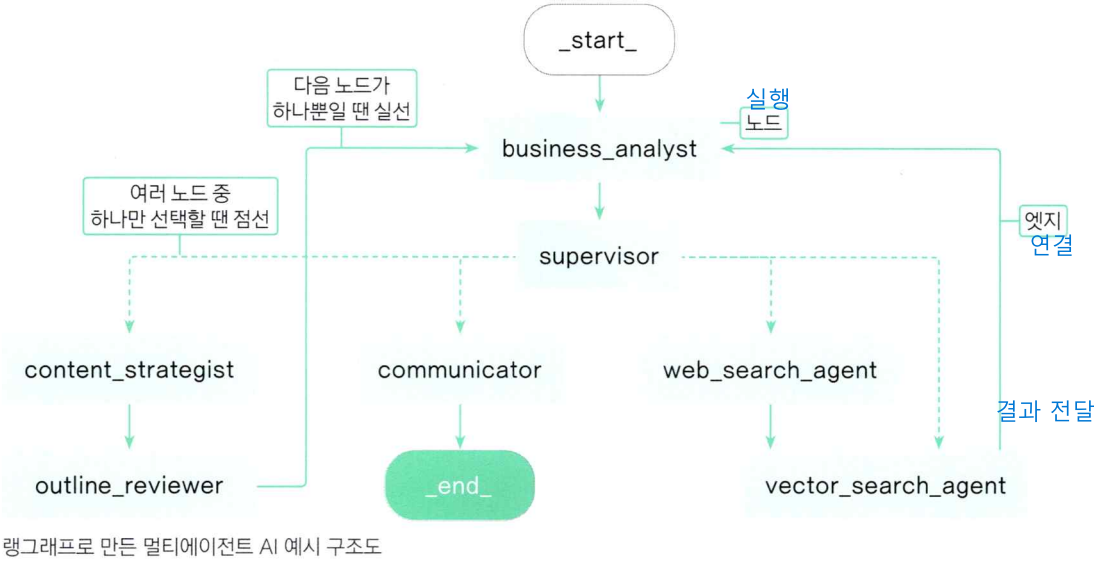
랭그래프의 기본 개념을 배우고, 랭그래프를 활용해 간단한 챗봇을 만들어 보겠습니다.

랭그래프란?

랭그래프LangGraph는 랭체인에서 한발 더 나아가 여러 AI 에이전트를 연결하여 상황에 맞게 다음 작업을 하도록 구성할 수 있게 해주는 프레임워크입니다. 랭그래프를 처음 접하는 사람은 '그래프'라는 용어 때문에 마우스로 드래그 앤 드롭하여 워크플로를 구성하는 시각화 도구로 오해할 수 있는데, 이는 수학의 그래프 이론에서 따온 개념입니다. 그래프 이론에서는 각각의 객체(요소)를 노드로, 노드 간의 연결을 엣지로 표현합니다. 랭그래프는 언어 모델이 처리해야 할 일의 단계와 순서를 그래프로 명확하게 지정하여 앞으로 어떻게 행동하고 판단할지 기준을 제시하게 해줍니다.

랭그래프의 기본 개념 — 노드, 엣지, 상태

다음은 앞으로 랭그래프를 사용하여 만들 책과 보고서 목차 쓰는 AI 에이전트의 구조도입니다. GPT에게 질문하면 한 번에 답을 제공하는 방식이 아니라 AI 에이전트가 스스로 판단하고 여러 단계를 거쳐 작업을 수행하도록 설계되어 있습니다. 랭그래프에서는 이런 작업의 흐름을 노드, 엣지, 상태를 활용해 표현합니다.



랭그래프로 만든 멀티에이전트 AI 예시 구조도

노드node는 하나의 작업이나 단계를 나타냅니다. 다음 그림에서 'business_analyst', 'supervisor'처럼 네모 상자로 표시한 작업 요소가 노드이고, 노드의 연결을 나타내는 화살표는 엣지edge입니다. 하나의 작업이 끝났을 때 다음 작업으로 무엇을 할지는 상황에 따라 다를 수 있습니다. 한 노드에서 연결되는 다음 노드가 하나뿐인 경우에는 실선으로, 작업 결과에 따라 여러 노드 중에서 하나를 선택할 수 있는 경우에는 점선으로 표현했습니다. 그리고 상태state는 노드가 작업한 결과를 기록해 두는 작업 일지를 의미합니다. 이 절에서는 간단한 챗봇을 만들면서 노드, 엣지, 상태의 개념을 자세히 배우겠습니다.

처음부터 복잡한 랭그래프를 구현하기는 어려우니 우선 간단한 예제로 시작합시다.

Do it! 실습 랭그래프로 간단한 챗봇 만들기

결과 파일: chap12/sec01/langgraph_simple_chatbot.ipynb

실제로 코딩하며 랭그래프의 차이를 직접 경험해 봅시다. 사용자와 대화를 주고받는 단순한 구조의 챗봇을 만들겠습니다. 이 예제로는 랭그래프의 장점을 제대로 느끼기 어렵겠지만 랭그래프의 기본 구조를 이해하는 데 도움될 것입니다. 12-3절과 그 이후 실습에서 랭그래프의 기능을 점점 더 확장해 보겠습니다.

✦ 이번 실습에서 사용하는 예제 코드는 랭그래프 공식 문서인 LangGraph Quick Start(https://langchain-ai.github.io/langgraph/tutorials/introduction/)의 내용을 참고했습니다.

1. 랭그래프의 기능을 단계별로 이해하기 위해 주피터 노트북 파일로 실습하겠습니다. langgraph_simple_chatbot.ipynb 파일을 새로 만들고 다음과 같이 입력해 랭그래프를 설치합니다.

랭그래프 설치하기 langgraph_simple_chatbot.ipynb (1)

```
%pip install langgraph
```

nb:노트북
비동기처리 error
py에서 개발하는 게..

2. 다음과 같이 선언해 GPT 모델을 설정합니다.

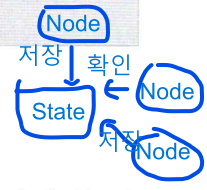
GPT 모델 설정하기 langgraph_simple_chatbot.ipynb (2)

```
from langchain_openai import ChatOpenAI

# 모델 초기화
model = ChatOpenAI(model="gpt-4o-mini")
model.invoke('안녕하세요!')
```

이 셀을 실행하면 다음처럼 GPT의 답변이 잘 출력됩니다.

```
AIMessage(content='안녕하세요! 어떻게 도와드릴까요?',
(... 생략 ...)
```



LangGraph에서 가장 중요.

Do it! 실습 **상태 정의하기** 상태: 저장소 처리되는 것들에 대한 저장소.

처리->결과->저장

결과 파일: sec01/langgraph_simple_chatbot.ipynb

랭그래프에서 상태는 언어 모델이 임무를 수행하면서 현재 상태를 명확히 관리할 수 있도록 돕는 요소입니다. 랭그래프는 여러 노드, 즉 AI 에이전트가 각자 맡은 일을 수행하도록 구성되고 이 노드들은 상황에 맞게 작업할 수 있도록 필요한 정보를 주고받아야 합니다.
일반적으로 랭그래프에서는 State(상태) 클래스에 필요한 데이터 형식을 최대한 명확하게 정의합니다. 이렇게 정의된 상태에 각 노드에서 처리된 결과를 저장하고 이 정보를 다음 작업을 수행할 노드로 전달합니다. 이처럼 랭그래프에서는 각 작업을 진행하는 데 필요한 항목들의 자료 형태를 상태에 미리 정해 놓아야 각 노드가 필요한 정보를 정확히 받아 작업이 순차로 잘 이어집니다.

대화 messages 아니라 상태(state)에 저장하면 됨.

이전 실습에서 만든 챗봇에 State 클래스와 그래프를 생성해 보겠습니다.

상태 정의하기 langgraph_simple_chatbot.ipynb (3)

```
from typing import Annotated  # annotated는 타입 힌트를 사용할 때 사용하는 함수
from typing_extensions import TypedDict  # TypedDict는 딕셔너리 타입을 정의할 때 사용하는 함수

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
```

```
class State(TypedDict):—1
    """
    State 클래스는 TypedDict를 상속받습니다.

    속성:
        messages (Annotated[list[str], add_messages]): 메시지들은 "list" 타입을 가집니다.—2
        'add_messages' 함수는 이 상태 키가 어떻게 업데이트되어야 하는지를 정의합니다.—3
        (이 경우, 메시지를 덮어쓰는 대신 리스트에 추가합니다)
    """
    messages: Annotated[list[str], add_messages]—2

# StateGraph 클래스를 사용하여 State 타입의 그래프 생성
graph_builder = StateGraph(State)—4
```

1. State 클래스는 TypedDict를 사용하여 딕셔너리 형태로 관리됩니다.
2. State 클래스에는 messages라는 변수만 포함되어 있으며, 이 변수는 Annotated를 사용해 문자열로 구성된 리스트 형식임을 명시합니다.
3. add_messages 함수를 추가합니다. add_messages 함수는 langgraph에서 제공하는 함수로 문자열이 주어질 때 이를 추가하는 기능을 합니다.
4. 생성한 State를 이용해 StateGraph를 만들어 graph_builder라는 변수에 담습니다.

앞으로 이렇게 만든 graph_builder에 노드와 엣지들을 연결할 예정입니다.

Do it! 실습 노드 생성하기

결과 파일: sec01/langgraph_simple_chatbot.ipynb

노드를 설정할 차례입니다. 앞서 설명했듯이 랭그래프는 각 노드에서 처리한 결과를 상태에서 관리하고, 각 노드를 엣지로 연결하는 그래프 형태로 표현하여 대화나 데이터 흐름을 관리합니다. 여기서 노드는 그래프의 한 지점을 의미하며 하나의 단계를 표현한다고 이해하면 쉽습니다.

사용자가 질문하면 답변을 생성하는 generate라는 노드를 가진 랭그래프를 만들겠습니다. 이 노드는 기존의 대화 내용에 기반하여 GPT로 답변을 생성하는 역할을 합니다.

노드 생성하기 langgraph_simple_chatbot.ipynb (4)

```
def generate(state: State):
    """
    주어진 상태를 기반으로 챗봇의 응답 메시지를 생성합니다.

    매개변수:
    state (State): 현재 대화 상태를 나타내는 객체로, 이전 메시지들이 포함되어 있습니다.

    반환값:
    dict: 모델이 생성한 응답 메시지를 포함하는 딕셔너리.
          형식은 {"messages": [응답 메시지]}입니다.
    """
    return {"messages": [model.invoke(state["messages"])]}

graph_builder.add_node("generate", generate)
```

1. generate 함수를 만듭니다. 매개변수는 앞서 정의한 State를 받도록 설정합니다.
2. state 안에는 messages라는 리스트를 담을 수 있는 변수만 포함되어 있습니다. 이 노드는 GPT 모델에 state["messages"]를 전달하여 답변을 받아 온 뒤 그 결과를 딕셔너리 형태로 반환하는 간단한 역할을 합니다.
3. 마지막으로 앞서 만든 graph_builder에 generate 노드를 추가합니다. 이때 노드의 이름과 그 해당 함수명을 함께 넣어 줍니다.

이 셀을 실행하면 노드가 생성됩니다.

```
<langgraph.graph.state.StateGraph at 0x209c230cd70>
```

Do it! 실습 엣지 설정하기

결과 파일: sec01/langgraph_simple_chatbot.ipynb

그래프는 노드와 엣지로 구성됩니다. 이번 예제에서는 노드 앞뒤로 START와 END 노드를 지정하고 엣지를 설정해 보겠습니다. START와 END는 랭그래프에서 제공하는 노드입니다.

1. 앞서 만든 graph_builder에 연결 관계를 선언합니다. 그래프는 START에서 챗봇을 거쳐 END로 가는 흐름으로 구성됩니다. 이 흐름을 정의하고 그래프를 컴파일하여 graph로 선언합니다.

graph 선언하기 langgraph_simple_chatbot.ipynb (5)

```
graph_builder.add_edge(START, "generate")
graph_builder.add_edge("generate", END)

graph = graph_builder.compile()
```

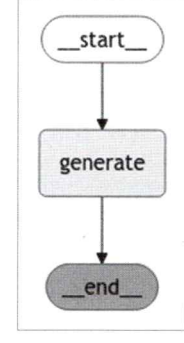
2. 현재 주피터 노트북에서 진행하고 있으므로 우리가 만든 그래프의 구조를 파이썬으로 그려 볼 수 있습니다. 다음과 같이 입력하고 실행해 보세요.

그래프 도식화하기 langgraph_simple_chatbot.ipynb (6)

```
from IPython.display import Image, display

try:
    display(Image(graph.get_graph().draw_mermaid_png()))
except Exception:
    pass
```

이 셀을 실행하면 다음과 같이 구조가 간단한 챗봇 그래프가 출력 됩니다.



3. 그래프를 이용해 답변을 생성하기 위해 "messages"의 리스트에 문장을 넣습니다. 그리고 graph.invoke()의 결과 데이터 타입과 전체 response를 확인하는 코드를 작성합니다.

리스트에 문장 추가하기 langgraph_simple_chatbot.ipynb (7)

```
response = graph.invoke({"messages": ["안녕하세요! 저는 이성용입니다."]})

print(type(response))
response
```

이 셀을 실행해 보면 데이터가 AddableValuesDict 형태로 반환된 것을 알 수 있습니다. 이때 사용자가 입력한 문장이 담긴 리스트에 GPT의 답변이 추가된 상태로 결과가 반환됩니다. 이런 결과가 나온 이유는 State를 정의할 때 messages에 add_messages를 추가했기 때문입니다. 이렇게 설정한 덕분에 새로 입력된 메시지가 기존 메시지 리스트에 덧붙여지는 방식으로 동작합니다.

```
<class 'langgraph.pregel.io.AddableValuesDict'>

{'messages': [HumanMessage(content='안녕하세요! 저는 이성용입니다', additional_kwargs={},
response_metadata={}, id='fe2c6e52-ec4a-403b-b585-bcd539d55686'),
AIMessage(content='안녕하세요, 이성용님! 만나서 반갑습니다. 어떻게 도와드릴까요?',
(... 생략 ...)
```

4. 이전 대화 내용을 이어 나가려면 다음과 같이 .append를 사용하여 원하는 메시지를 추가한 뒤, graph.invoke로 다음 문장을 생성합니다.

이전 대화 내용에 새 메시지 추가하기 langgraph_simple_chatbot.ipynb (8)

```
response["messages"].append("제 이름을 아시나요?")
graph.invoke(response)
```

이 셀을 실행하면 다음처럼 대화 내용을 유지한 상태로 답변을 생성해 줍니다. 앞에서 말했던 제 이름을 잘 기억하고 대답하네요.

```
{'messages': [HumanMessage(content='안녕하세요! 저는 이성용입니다', ... 생략 ...,
AIMessage(content='안녕하세요, 이성용님! 만나서 반갑습니다. 어떻게 도와드릴까요?',
(... 생략 ...)
  HumanMessage(content='제 이름을 아시나요?', ... 생략 ...,
  AIMessage(content='네, 이성용님이라고 말씀해주셨습니다. 더 궁금한 점이나 이야기하고 싶은 내용이
있으면 언제든지 말씀해 주세요!', ... 생략 ...})]}
```

단순한 챗봇을 만드는데 왜 이렇게 복잡하게 구성해야 하는지 의문이 생길 수 있습니다. 하지만 랭그래프 방식으로 구성하면 앞으로 언어 모델에게 더 복잡한 작업을 맡길 때 확장하기 쉽고 명확해집니다.

Do it! 실습 스트림 출력하기

결과 파일: sec01/langgraph_simple_chatbot.ipynb

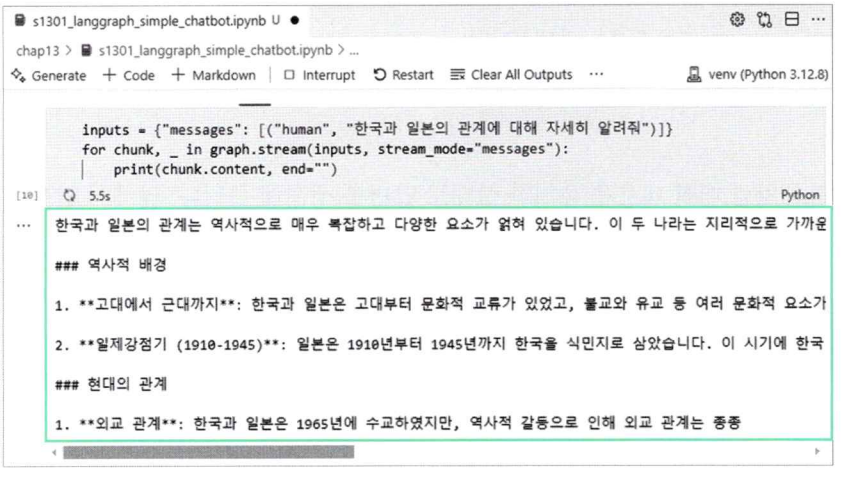
언어 모델의 반응 속도를 크게 신경 쓰지 않아도 되는 문서 생성 작업에서는 현재 방식을 사용해도 아무 문제가 없습니다. 하지만 챗봇을 만들려면 사용자의 질문에 최대한 빠르게 반응할 수 있도록 스트림 방식으로 출력해야 합니다. 이전 파일에 이어서 코드를 작성해 보겠습니다.

스트림 방식으로 출력할 때에는 `.invoke` 대신 `.stream`을 사용합니다. 이때 `stream_mode`를 `messages`로 선택하면 메시지를 스트림 방식으로 실시간 출력합니다.

스트림 방식으로 출력하기 langgraph_simple_chatbot.ipynb (9)

```
inputs = {"messages": [("human", "한국과 일본의 관계에 대해 자세히 알려 줘.")]}
for chunk, _ in graph.stream(inputs, stream_mode="messages"):
    print(chunk.content, end="")
```

이 셀을 실행하면 GPT가 답변을 생성하는 대로 화면에 순차적으로 출력해 줍니다.



한 걸음 더! 다양한 스트림 모드 알아보기

스트림 모드는 앞 실습에서 사용한 messages뿐만 아니라 values, updates, debug, custom 등 다양한 옵션이 있습니다.

✦ 스트림 모드를 더 자세히 알고 싶다면 랭그래프 공식 문서를 참고하세요(https://langchain-ai.github.io/langgraph/how-tos/streaming/?h=stream+mode).

옵션	설명
messages	메시지를 스트림 방식으로 실시간 출력합니다.
values	랭그래프의 단계별로 상태가 어떻게 변하는지 파악할 때 사용합니다.
updates	단계별로 변경된 내용만 반환합니다.
debug	디버깅용 옵션으로 실행되는 과정의 정보를 자세히 제공합니다.
custom	사용자 정의 방식으로 스트림을 설정할 수 있습니다.

12-2 대화 내용을 저장하는 메모리

12-1절에서 만든 챗봇이 이전 대화 내용을 기억하게 하려면 매번 수동으로 업데이트해야 했습니다. 이런 방식이 필요한 경우도 있지만 특수한 상황이 아니라면 랭그래프에서 제공하는 메모리Memory를 활용해 대화 내용을 간편하게 저장할 수 있습니다.

Do it! 실습 랭그래프의 메모리 기능 활용하기

결과 파일: sec02/langgraph_memory.py

1. 랭그래프의 메모리를 다루기 전에 기본 챗봇을 만들겠습니다. langgraph_memory.py 파일을 새로 만들고 다음처럼 코드를 입력합니다. 코드 중 일부는 12-1절에서 만든 챗봇 코드를 그대로 가져왔습니다.

메모리를 적용하기 전 기본 챗봇 만들기 — langgraph_memory.py

```
from langchain_openai import ChatOpenAI

# 모델 초기화
model = ChatOpenAI(model="gpt-4o-mini")

from typing import Annotated  # annotated는 타입 힌트를 사용할 때 사용하는 함수
from typing_extensions import TypedDict

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class State(TypedDict):
    """
    State 클래스는 TypedDict를 상속받습니다.

    속성:
        messages (Annotated[list[str], add_messages]): 메시지들은 "list" 타입을 가집니다.
        주석에 있는 'add_messages' 함수는 이 상태 키가 어떻게 업데이트되어야 하는지를 정의합니다.
        (이 경우, 메시지를 덮어쓰는 대신 리스트에 추가합니다)
    """
    messages: Annotated[list[str], add_messages]
```

```
# StateGraph 클래스를 사용하여 State 타입의 그래프 생성
graph_builder = StateGraph(State)

#-----------
def generate(state: State):
    """
    주어진 상태를 기반으로 챗봇의 응답 메시지를 생성합니다.

    매개변수:
    state (State): 현재 대화 상태를 나타내는 객체로, 이전 메시지들이 포함되어 있습니다.

    반환값:
    dict: 모델이 생성한 응답 메시지를 포함하는 딕셔너리.
          형식은 {"messages": [응답 메시지]}입니다.
    """
    return {"messages": model.invoke(state["messages"])}

graph_builder.add_node("generate", generate)

graph_builder.add_edge(START, "generate")
graph_builder.add_edge("generate", END)

graph = graph_builder.compile()

#------------ 여기서부터 달라진 코드가 있음
from langchain.schema import HumanMessage

while True:
    user_input = input("You\t:")

    if user_input in ["exit", "quit", "q"]:
        break

    for event in graph.stream({"messages": [HumanMessage(user_input)]}, stream_
mode="values"):
        event["messages"][-1].pretty_print()

    print(f'\n현재 메시지 개수: {len(event["messages"])}\n------------------\n')
```

질문

1. while 반복문을 살펴보겠습니다. 파이썬의 input 함수를 사용하여 터미널 창에서 사용자가 입력한 내용을 문자열로 받아 user_input 변수에 저장합니다. 이때 받은 값이 exit, quit, q 중 하나라면 대화를 종료합니다.
2. user_input이 유효한 값이라면 graph.stream의 messages에 HumanMessage 클래스를 이용해 리스트 형태로 담아 실행합니다. 이때 stream_mode를 messages가 아니라 values로 설정하여 각 단계의 상태 변화를 스트림 방식으로 가져옵니다. 스트림 방식이므로 for 문을 이용해 메시지를 하나씩 가져올 수 있습니다.
3. event["messages"]에는 현재 상태의 메시지들이 저장되어 있으므로 가장 마지막 메시지를 .pretty_print()로 터미널 창에 출력합니다. .pretty_print()는 객체의 내용을 보기 좋게 출력할 때 사용하며, 출력하는 메시지가 AIMessage인지 HumanMessage인지에 따라 터미널 창에 자동으로 구분하여 표시해 줍니다.
4. 마지막으로 현재 event["messages"]에 저장된 메시지의 개수를 출력하고 while 반복문은 다시 처음으로 돌아갑니다.

코드를 실행하고 과거 대화에 기반해서 대화를 이어 갈 수 있는지 터미널 창에서 테스트해 봅시다. 메시지 개수가 2개에서 더 이상 늘어나지 않고, 매번 새로운 대화로 인식해서 바로 앞에서 말해 준 제 이름도 기억하지 못합니다.

```
You      :난 이성용이야.
====== Human Message =======

난 이성용이야.
======== Ai Message ========

안녕하세요, 이성용님! 어떻게 도와드릴까요?

현재 메시지 개수: 2
--------------------

You      :내 이름이 뭐지?
====== Human Message =======

내 이름이 뭐지?
======== Ai Message ========

죄송하지만, 당신의 이름을 알 수 있는 정보가 없습니다. 이름을 알려주시면 좋겠습니다!

현재 메시지 개수: 2
--------------------

You      :q
```

2. 기존 코드에서 단 몇 줄만 바꿔 메모리를 추가하면 이전 대화 내용을 기억한 상태로 대화를 이어 나갈 수 있도록 설정할 수 있습니다. 코드를 다음처럼 수정합니다.

메모리 추가하기 langgraph_memory.py

```
(... 생략 ...)

graph_builder.add_node("generate", generate)

graph_builder.add_edge(START, "generate")
graph_builder.add_edge("generate", END)

from langgraph.checkpoint.memory import MemorySaver
memory = MemorySaver()

config = {"configurable": {"thread_id": "abcd"}}

graph = graph_builder.compile(checkpointer=memory)

#------------
from langchain.schema import HumanMessage

while True:
    user_input = input("You\t:")

    if user_input in ["exit", "quit", "q"]:
        break

     for event in graph.stream({"messages": [HumanMessage(user_input)]}, config,
stream_mode="values"):
        event["messages"][-1].pretty_print()

    print(f'\n현재 메시지 개수: {len(event["messages"])}\n--------------------\n')
```

1. 랭그래프에서 제공하는 MemorySaver를 임포트하고 memory라는 이름의 객체로 만듭니다.
2. memory 객체는 graph_builder.compile(checkpointer=memory)에서 설정되어 대화 내용을 계속 쌓아 갈 수 있도록 만듭니다.
3. config에서 thread_id를 abcd로 설정합니다. 이때 thread_id는 임의의 문자열로 설정합니다. config의 thread_id는 일종의 대화방 ID라고 생각하면 됩니다. while 문으로 메시지를 여러 번 반복해서 입력하더라도 thread_id가 유지된다면 기존 대화 내용을 계속 보존한 상태로 진행할 수 있습니다.

이 코드를 실행해 보면 기존 대화 내용을 계속 쌓아 나가는 것을 알 수 있습니다. 새로운 메시지를 입력해도 지난 대화 내용에 기반해서 답변을 잘 생성합니다.

```
You      :난 이성용이야.
====== Human Message =======

난 이성용이야.
======== Ai Message ========

안녕하세요, 이성용님! 어떻게 도와드릴까요?

현재 메시지 개수: 2
--------------------

You      :내 이름이 뭐지?
====== Human Message =======

내 이름이 뭐지?
======== Ai Message ========

당신의 이름은 이성용입니다. 다른 질문이나 요청이 있으신가요?

현재 메시지 개수: 4
--------------------

You      :q
```

12-3 인터넷 검색 후 기사를 작성하는 챗봇 만들기

tool

LLM

07장에서는 오픈AI의 GPT API를 활용할 때 펑션 콜링을 사용했고, 08장에서는 랭체인에서 도구 호출하는 방법을 배웠습니다. 랭그래프는 랭체인에 기반하므로 GPT와 같은 언어 모델이 미리 정의한 도구를 사용할 수 있게 구현할 수 있습니다. 이번 실습에서는 사용자가 1 주제를 제시 3 하면 인터넷을 검색 2 하여 최신 이슈를 기반으로 세부 주제를 선정하고, 그 주제를 검색해서 최종 기사를 작성하는 신문기자 챗봇을 만들겠습니다.

Do it! 실습 신문기자 챗봇 만들기

결과 파일: sec03/langgraph_tools.ipynb

1. 먼저 기사를 어떻게 작성할지 일의 순서를 설명합니다. 여기서 {about}에 주제만 입력하면 챗봇이 알아서 기사를 쓰도록 만들어 보겠습니다.

```
너는 신문기자이다.
최근 {about}에 대해 비판하는 심층 분석 기사를 쓰려고 한다.

- 최근 어떤 이슈가 있는지 검색하고 사람들이 제일 관심있어 할만한 주제를 선정하고 왜 선정했는지 말해줘.
- 그 내용으로 원고를 작성하기 위한 목차를 만들고 목차 내용을 채우기 위해 추가로 검색할 내용을 리스트로 정리해봐.
- 검색할 리스트를 토대로 재검색해.
- 목차에 있는 내용을 작성하기 위해 더 검색이 필요한 정보가 있는지 확인하고 있다면 추가로 검색해.
- 검색된 결과에서 원하는 정보를 찾지 못했다면 다른 검색어로 재검색해도 좋아.

더 이상 검색할 내용이 없다면 조선일보 신문 기사 형식으로 최종 기사를 작성한다.
제목, 부제, 리드문, 본문의 구성으로 작성한다. 본문 내용은 심층 분석 기사에 맞게 구체적이고 깊이 있게 작성해야 한다.
```

현재 시간을 알려 주는 함수와 웹 검색을 하는 함수를 랭체인의 도구로 등록하고 랭그래프로 구현하겠습니다.

2. langgraph_tools.ipynb 파일을 생성하고 언어 모델을 GPT로 설정합니다.

언어 모델 설정하기 — langgraph_tools.ipynb (1)

```
from langchain_openai import ChatOpenAI

# 모델 초기화
model = ChatOpenAI(model="gpt-4o", temperature=0.01)
model.invoke('안녕하세요!')
```

언어 모델이 잘 설정되었는지 테스트해 보기 위해 문장을 생성해 봤습니다. GPT가 잘 설정되었네요.

```
AIMessage(content='안녕하세요! 어떻게 도와드릴까요?',
(... 생략 ...)
```

3. 랭그래프의 상태를 선언하고 그 안에 필요한 정보들을 담는 코드를 작성합니다. 이 코드는 12-1절과 12-2절에서 랭그래프를 사용한 코드와 동일합니다.

상태 설정하기 — langgraph_tools.ipynb (2)

```
from typing import Annotated            # annotated는 타입 힌트를 사용할 때 사용하는 함수
from typing_extensions import TypedDict  # TypedDict는 딕셔너리 타입을 정의할 때 사용하는 함수

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class State(TypedDict):
    """
    State 클래스는 TypedDict를 상속받습니다.

    속성:
        messages (Annotated[list[str], add_messages]): 메시지들은 "list" 타입을 가집니다.
        주석에 있는 'add_messages' 함수는 이 상태 키가 어떻게 업데이트되어야 하는지를 정의합니다.
        (이 경우, 메시지를 덮어쓰는 대신 리스트에 추가합니다)
    """
    messages: Annotated[list[str], add_messages]

# StateGraph 클래스를 사용하여 State 타입의 그래프 생성
graph_builder = StateGraph(State)
```

랭그래프에서도 랭체인과 마찬가지로 도구를 정의하고 활용할 수 있습니다. 08-3절과 10-4절에서 랭체인 도구를 다룰 때 사용한 시간을 알려 주는 get_current_time 함수와 웹 검색을 해주는 get_web_search 함수를 활용해 코드를 작성해 보겠습니다.

4. `get_web_search` 함수를 도구로 등록해 보겠습니다. 10-4절의 streamlit_with_web_search.py 파일 코드를 참고하여 작성했습니다.

도구 등록하기 — langgraph_tools.ipynb (3)

```
from langchain_core.tools import tool
from datetime import datetime
import pytz
from langchain_community.tools import DuckDuckGoSearchResults
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper

import bs4
from langchain_community.document_loaders import WebBaseLoader

# 도구 함수 정의
@tool ③
def get_current_time(timezone: str, location: str) -> str:
    """현재 시각을 반환하는 함수."""
    try:
        tz = pytz.timezone(timezone)
        now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
        result = f'{timezone} ({location}) 현재 시각 {now}'
        # print(result)
        return result
    except pytz.UnknownTimeZoneError:
        return f"알 수 없는 타임존: {timezone}"

@tool ③
def get_web_search(query: str, search_period: str='m') -> str: ①
    """
    웹 검색을 수행하는 함수.

    Args:
        query (str): 검색어
        search_period (str): 검색 기간 (e.g., "w" for past week (default), "m" for past
month, "y" for past year, "d" for past day)

    Returns:
        str: 검색 결과
```

```
    """
    wrapper = DuckDuckGoSearchAPIWrapper(
        # region="kr-kr",—②
        time=search_period
    )

    print('\n-------- WEB SEARCH --------')
    print(query)
    print(search_period)

    search = DuckDuckGoSearchResults(
        api_wrapper=wrapper,
        # source="news",—②
        results_separator=';\n'
    )

    searched = search.invoke(query)

    for i, result in enumerate(searched.split(';\n')):
        print(f'{i+1}. {result}')

    return searched

# 도구 바인딩
tools = [get_current_time, get_web_search]
```

① 함수 get_web_search의 search_period 매개변수에 기본값으로 최근 1개월을 의미하는 'm'을 설정합니다.

② 검색 언어와 지역을 의미하는 region="kr-kr"과 뉴스 검색만 설정하는 source="news"는 주석으로 처리해 더 자유롭게 검색할 수 있도록 합니다.

③ 이 함수를 랭체인과 랭그래프의 언어 모델에 연결하려면 @tool 데코레이터를 함수 위에 붙여서 tool로 등록해야 합니다. 이렇게 등록된 도구들은 tools에 리스트 형태로 담아 둡니다.

5. tools에 바인딩한 도구들이 랭체인과 랭그래프 방식으로 잘 작동하는지 확인해 봅시다. tools[0]은 시간을 알려 주는 get_current_time 함수를 의미합니다. 여기에 필요한 매개변수는 딕셔너리로 입력합니다.

현재 시각을 얻는 도구 get_current_time 실행 테스트하기 — langgraph_tools.ipynb (4)

```
tools[0].invoke({"timezone": "Asia/Seoul", "location": "서울"})
```

이 셀을 실행하면 질문한 서울의 현재 시각을 잘 답변합니다.

```
'Asia/Seoul (서울) 현재 시각 2024-11-25 22:08:45'
```

6. tools[1]은 웹 검색을 하는 get_web_search 함수를 의미합니다. '파이썬'을 질의어로 하고, 검색 기간을 의미하는 search_period를 'm'으로 설정해서 최근 한 달 간의 문서를 검색합니다.

웹 검색 도구 get_web_search 실행 테스트하기 — langgraph_tools.ipynb (5)

```
tools[1].invoke({"query": "파이썬", "search_period": "m"})
```

이 셀을 실행하면 다음과 같이 파이썬에 대해 잘 검색합니다.

```
-------- WEB SEARCH --------
파이썬
m
1. snippet: 그동안 공부한 내용을 정리하는 차원에서 포스팅을 하려고 합니다. 우선 아래의 목차대로 진행하고 중간에 추가할 내용이 있으면 추가하는 방식으로 1. 초급 (Python 기초)파이썬 소개 및 개발 환경 구축파이썬이란?파이썬 설치 및 실행 방법IDE(예: VS Code, PyCharm) 설정 (처음에는 PyCharm에서 코딩하다가 ..., title: 파이썬(Python) 시작하기, link: https://gotoinfo.tistory.com/14
2. snippet: 파이썬 설치가 완료되었으니 실행해 봅니다. 파이썬 실행을 위해 윈도우 창에서 cmd를 입력하여 명령프롬프트 창을 엽니다. (단축키: 윈도우키 + R 누른 후 cmd 입력) 파이썬 실행을 위해 프롬프트 창에 python을 입력하면 파이썬이 실행되는 것을 확인할 수 있습니다., title: [Windows] Python 설치하는 방법, link: https://daily-canvas.tistory.com/2
(... 생략 ...)
```

7. tools가 어떻게 저장되었는지 확인하기 위해서 다음과 같이 for 문을 사용해 하나씩 출력해 보겠습니다.

도구 목록 확인하기 — langgraph_tools.ipynb (6)

```
for tool in tools:
    print(tool.name, tool)
```

다음처럼 도구 이름과 도구를 설명한 내용이 함께 정리되어 있습니다.

```
get_current_time name='get_current_time' description='현재 시각을 반환하는 함수.' args_
schema=<class 'langchain_core.utils.pydantic.get_current_time'> func=<function get_
current_time at 0x000002249C6E9620>
get_web_search name='get_web_search' description='웹 검색을 수행하는 함수.\n\nArgs:\n
query (str): 검색어\n    search_period (str): 검색 기간 (e.g., "w" for past week (default),
 "m" for past month, "y" for past year, "d" for past day)\n\nReturns:\n    str: 검색 결
과' args_schema=<class 'langchain_core.utils.pydantic.get_web_search'> func=<function
get_web_search at 0x000002249C97B9C0>
```

8. 이제 챗봇의 응답 메시지를 생성하는 generate 노드를 만들겠습니다.그리고 model로 선언한 언어 모델에 .bind_tools로 붙인 model_with_tools로 도구를 사용합니다. 이렇게 만든 generate 함수를 graph_builder에 "generate"라는 이름의 노드로 붙입니다.

✦ 언어 모델에 도구를 연결할 때 .bind_tools를 사용하는 내용은 10-4절의 streamlit_with_web_search.py 파일을 작성할 때 다루었습니다.

언어 모델에 .bind_tools로 사용할 도구 연결하기 — langgraph_tools.ipynb (7)

```
model_with_tools = model.bind_tools(tools) # GPT 언어 모델에 도구 연결

def generate(state: State):
    """
    주어진 상태를 기반으로 챗봇의 응답 메시지를 생성합니다.

    매개변수:
    state (State): 현재 대화 상태를 나타내는 객체로, 이전 메시지들이 포함되어 있습니다.

    반환값:
    dict: 모델이 생성한 응답 메시지를 포함하는 딕셔너리
          형식은 {"messages": [응답 메시지]}입니다.
    """
    return {"messages": model_with_tools.invoke(state["messages"])}

graph_builder.add_node("generate", generate)
```



9. 랭그래프에서 도구를 편리하게 사용하기 위해 `BasicToolNode` 클래스를 만들겠습니다. 지금 만들고 있는 랭그래프의 `generate` 노드에서 챗봇의 답변을 생성할 때 `get_web_search`나 `get_current_time`과 같은 도구를 사용해야 한다면 따로 독립된 노드에서 실행하게 됩니다. `BasicToolNode` 클래스는 `AIMessage`에서 도구 요청이 있을 때 이를 실행시키는 역할을 합니다.

✦ 다음 코드는 랭그래프 공식 문서: LangGraph Quick Start(https://langchain-ai.github.io/langgraph/tutorials/introduction/#part-2-enhancing-the-chatbot-with-tools)를 참고해서 작성했습니다.

도구를 사용하는 노드 클래스 생성하기 — langgraph_tools.ipynb (8)

```
import json
from langchain_core.messages import ToolMessage

class BasicToolNode:
    """
    (함수)
    도구를 실행하는 노드 클래스입니다. 마지막 AIMessage에서 요청된 도구를 실행합니다.
    Attributes:
        tools_by_name (dict): 도구 이름을 키로 하고 도구 객체를 값으로 가지는 사전입니다.
    Methods:
        __init__(tools: list): 도구 객체들의 리스트를 받아서 초기화합니다.
        __call__(inputs: dict): 입력 메시지를 받아서 도구를 실행하고 결과 메시지를 반환합니다.
    """
    """A node that runs the tools requested in the last AIMessage."""

    def __init__(self, tools: list) -> None:  ①
        self.tools_by_name = {tool.name: tool for tool in tools}

    def __call__(self, inputs: dict):  ②
        if messages := inputs.get("messages", []):
            # inputs에 messages가 있으면 messages를 가져오고 없으면 빈 리스트 가져오기
            message = messages[-1]
        else:
            raise ValueError("No message found in input")
        outputs = []
        for tool_call in message.tool_calls:  ③
            tool_result = self.tools_by_name[tool_call["name"]].invoke(
                tool_call["args"]
            )
            outputs.append(
                ToolMessage(
                    content=json.dumps(tool_result),
                    name=tool_call["name"],
                    tool_call_id=tool_call["id"],
```

FUNCTIONS = {'a':a, 'b':b}와 의미 동일
변수선언,초기화
실행
AIM -> tool_calls
결과 -> LLM
함수 이름
함수 객체

```
                )
            )                    기존      현재 실행 결과
        return {"messages": messages + outputs}

tool_node = BasicToolNode(tools=tools)
graph_builder.add_node("tools", tool_node)  ④
              노드 추가
```

① 클래스의 초기화 메서드 (__ init__)에 있는 tools_by_name은 딕셔너리 형태로 도구의 이름과 해당 도구 자체를 저장합니다.

② __call__ 메서드는 입력 메시지인 input을 딕셔너리 형태로 받습니다. 이때 input은 랭그래프에서 상태를 관리하는 state가 딕셔너리로 전달됩니다. state에는 messages가 포함되어 있습니다.

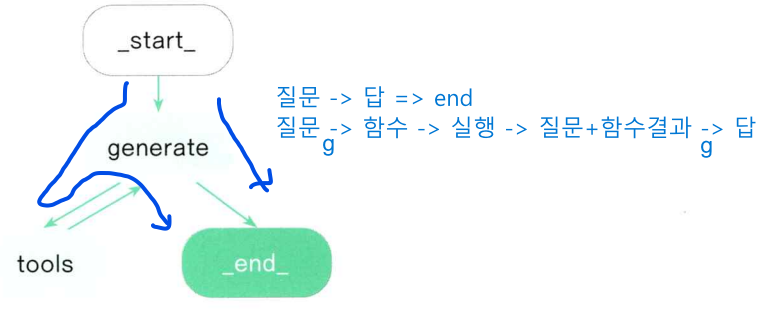
③ 이 messages의 가장 최근 메시지에 tool_calls가 존재하면 도구를 사용해야 하므로 self.tools_by_name[tool_call["name"]].invoke(tool_call["args"])를 for 문을 활용해 반복해서 실행합니다. tool_call["args"]에는 그 도구를 사용할 때 필요한 인잣값이 들어갑니다. 실행 결과는 tool_result에 저장됩니다. 이 tool_result를 outputs 리스트에 추가합니다.

④ 이렇게 만든 BasicToolNode 클래스를 tools라는 이름의 노드로 graph_builder에 추가합니다.

Do it! 실습 라우터 설정하기

결과 파일: sec03/langgraph_tools.ipynb

12-1절과 12-2절에서 만들었던 랭그래프의 흐름은 갈림길 없이 진행되었습니다. 이번 실습에서는 흐름이 조금 더 다양해집니다. 예를 들어 사용자가 질문하면 generate 노드에서 답변을 생성하고 그 후 바로 END 노드로 가서 작업을 끝낼 수도 있습니다. 하지만 만약 인터넷 검색이나 시간 확인이 필요하다고 판단되면 tools 노드에서 그 기능을 수행한 후 결과를 바탕으로 다시 generate 노드에 돌아와 답변을 생성할 수도 있습니다. 그리고 검색이나 시간 확인이 더 필요하다고 판단된다면 다시 tools 노드에서 작업을 반복할 수도 있습니다.



router: 분기점.

이번 실습에서는 AI 에이전트가 상황에 맞게 다음에 해야 할 일을 알아서 결정하도록 만들어 보겠습니다. AI 에이전트가 랭그래프 내에서 스스로 다음 경로를 선택해야 할 때 라우터router를 활용합니다. 상황에 따라 방향을 결정하는 것을 라우팅routing이라고하며 이때 조건에 따라 활성화되거나 비활성화되는 조건부 엣지conditional edge를 사용합니다.

✦ 라우터는 13-2절에서 자세히 알아봅니다.

1. START에서 generate까지는 이전과 같이 순차로 진행하고 generate 노드에서 언어 모델이 판단한 결과에 따라 경로가 달라지도록 설정합니다. 이를 위해 route_tools 함수를 만들고 state를 받아 messages 리스트에서 마지막 메시지를 확인합니다. 만약 마지막 메시지에 tool_calls가 포함되어 있으면 tools 노드로 이동하여 필요한 도구를 실행하고, 없다면 END 노드로 이동하여 작업을 종료합니다. tools 노드에서 작업을 마치면 다시 generate로 돌아옵니다.

Function

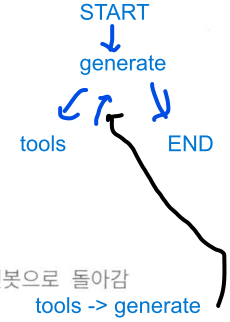
언어 모델이 도구 사용 여부를 판단하도록 라우터 설정하기 — langgraph_tools.ipynb (9)

```
def route_tools(state: State):
    """
    마지막 메시지에 도구 호출이 있는 경우 ToolNode로 라우팅하고,
    그렇지 않은 경우 끝으로 라우팅하기 위해 conditional_edge에서 사용합니다.
    """
    if isinstance(state, list):
        ai_message = state[-1]
    elif messages := state.get("messages", []):
        ai_message = messages[-1]
    else:
        raise ValueError(f"tool_edge 입력 상태에서 메시지를 찾을 수 없습니다: {state}")
    if hasattr(ai_message, "tool_calls") and len(ai_message.tool_calls) > 0:
        return "tools"
    return END

graph_builder.add_edge(START, "generate")

graph_builder.add_conditional_edges(
    "generate",
    route_tools,
    {"tools": "tools", END: END},
)
# 도구가 호출될 때마다 다음 단계를 결정하기 위해 챗봇으로 돌아감
graph_builder.add_edge("tools", "generate")
graph = graph_builder.compile()
```

BasicToolNode
최종 답변
조건에 따른 연결
함수 호출
특정 attr을 가지고 있니
함수 실행
함수 호출 존재
답변
에서
사용해서



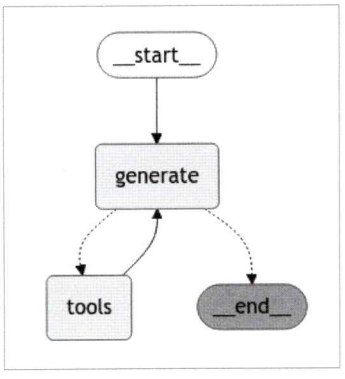
2. 실행 결과를 그래프로 그리려면 다음과 같이 코드를 입력합니다. 12-1절과 12-2절에서 그래프를 그릴 때 사용한 코드와 동일합니다.

그래프 출력하기 langgraph_tools.ipynb (10)

```
from IPython.display import Image, display

try:
    display(Image(graph.get_graph().draw_mermaid_png()))
except Exception:
    pass
```

셀을 실행하면 다음과 같이 그래프가 출력됩니다.



Do it! 실습 도구 테스트하고 기사 작성하기

결과 파일: sec03/langgraph_tools.ipynb

챗봇이 여러 도구 중에서 필요한 도구를 잘 찾아서 실행시키는지 테스트해 봅시다. 이어서 웹 검색 후 기사 작성 기능을 구현해 봅시다. 이때 '인터넷 검색해서 자료를 모으고 그걸로 기사 써줘'라고 지시할 수도 있지만 주제를 명확히 제시하는 것이 좋습니다. 어떤 내용을 작성해야 할지 순서대로 명시해야 언어 모델이 원하는 결과물을 만들어 낼 가능성이 커집니다. 언어 모델은 단순히 가이드에 따라 문장을 생성하지만, 주제와 순서를 명확히 제시하면 생성해야 할 다음 문장들을 목적에 부합하게 작성할 수 있으니까요.

1. messages에 HumanMessage로 '지금 서울 몇 시야?'라는 질문을 넣어 graph.stream으로 실행합니다. 스트림 출력을 위해 graph.invoke가 아닌 .stream으로 설정하고 stream_mode는 'messages'로 설정합니다. 결과가 AIMessageChunk로 나오자마자 for 문을 통해 즉각 출력됩니다. AIMessageChunk는 조각난 상태로 조금씩 넘어오므로 gathered라는 변수에 조각들을 계속 붙여 나가도록 했습니다. 이렇게 완성한 최종 결과 gathered를 출력합니다.

답변 출력하기 langgraph_tools.ipynb (11)

```
from langchain_core.messages import AIMessageChunk, HumanMessage

inputs = [HumanMessage(content="지금 서울 몇 시야?")]

gathered = None

for msg, metadata in graph.stream({"messages": inputs}, stream_mode="messages"):
    if isinstance(msg, AIMessageChunk):
        print(msg.content, end='')

        if gathered is None:
            gathered = msg
        else:
            gathered = gathered + msg

gathered
```

다음은 실행 결과입니다. 출력된 gathered 내용을 보면 tool_calls에 get_current_time 함수를 사용했고 arguments는 'Asia/Seoul', '서울' 이라고 기록되어 있습니다.

```
현재 서울 시간은 2024년 11월 26일 00시 01분입니다.

AIMessageChunk(content='지금 서울의 현재 시각은 2024년 11월 26일 00시 05분입니다.', additional_
kwargs={'tool_calls': [{'index': 0, 'id': 'call_7nt1a2vTBdrycIdPqia9yGKu', 'function'
: {'arguments': '{"timezone":"Asia/Seoul","location":"서울"}', 'name': 'get_current_
time'}, 'type': 'function'}]}, response_metadata={'finish_reason': 'tool_callsstop'
, 'model_name': 'gpt-4o-2024-08-06gpt-4o-2024-08-06', 'system_fingerprint': 'fp_7f6
be3efb0fp_7f6be3efb0'}, id='run-c37f070f-05c1-4a2a-b6de-67f0da5abdf4', tool_
calls=[{'name': 'get_current_time', 'args': {'timezone': 'Asia/Seoul', 'location': '
서울'}, 'id': 'call_7nt1a2vTBdrycIdPqia9yGKu', 'type': 'tool_call'}], tool_call_
chunks=[{'name': 'get_current_time', 'args': '{"timezone":"Asia/Seoul","location":"서
울"}', 'id': 'call_7nt1a2vTBdrycIdPqia9yGKu', 'index': 0, 'type': 'tool_call_chunk'}])
```

2. 앞에서는 inputs에 HumanMessage를 사용했지만 이번에는 언어 모델에게 역할을 지시하기 위해 SystemMessage를 사용합니다. 그리고 f-string에 기사 주제를 사용하여 {about}을 받아 오는 형태로 프롬프트를 입력합니다. 여기에서는 '서울 월드컵 경기장 잔디 문제'라는 주제를 설정했습니다.

프롬프트 설정하고 기사 작성하기 langgraph_tools.ipynb (12)

```
from langchain_core.messages import AIMessageChunk, SystemMessage

about = "서울 월드컵 경기장 잔디 문제"

inputs = [SystemMessage(content=f"""
너는 신문기자이다.
최근 {about}에 대해 비판하는 심층 분석 기사를 쓰려고 한다.

- 최근 어떤 이슈가 있는지 검색하고, 사람들이 제일 관심있어 할만한 주제를 선정하고, 왜 선정했는지 말해줘.
- 그 내용으로 원고를 작성하기 위한 목차를 만들고, 목차 내용을 채우기 위해 추가로 검색할 내용을 리스트로 정리해봐.
- 검색할 리스트를 토대로 재검색해.
- 목차에 있는 내용을 작성하기 위해 더 검색이 필요한 정보가 있는지 확인하고, 있다면 추가로 검색해.
- 검색된 결과에 원하는 정보를 찾지 못했다면 다른 검색어로 재검색해도 좋아.

더 이상 검색할 내용이 없다면, 조선일보 신문 기사 형식으로 최종 기사를 작성한다.
제목, 부제, 리드문, 본문의 구성으로 작성한다. 본문 내용은 심층 분석 기사에 맞게 구체적이고 깊이 있게 작성해야 한다.

""")]

for msg, metadata in graph.stream({"messages": inputs}, stream_mode="messages"):
    if isinstance(msg, AIMessageChunk):
        print(msg.content, end='')
```

실행 결과는 다음과 같습니다. 웹 검색으로 주제에 대한 최근 소식을 검색해서 기사의 방향을 결정하고 목차를 생성한 후, 관련 내용을 다시 검색합니다. 그리고 수집한 내용을 기반으로 기사를 작성했습니다.

```
-------- WEB SEARCH --------
서울 월드컵 경기장 잔디 문제
m
1. snippet: 서울 마포구 상암동 서울월드컵경기장의 잔디가 계절 변화가 뚜렷한 한국의 기후에서도 잘 자라도록 연구해야 할 서울시와 서울시설공단이 관련 해외사례 연구를 한 차례도 하지 않은 것으로 나타났다. 문성호 국민의힘 서울시의원은 6일 "공단을 대상으로 (잔디 관련) 해외사례 연구 및 관련 용역 ..., title: 상암 잔디 문제 날씨 탓하더니?…서울시 '해외 잔디' 연구 한 번도 안 해 - 경향신문, link: https://www.khan.co.kr/local/Seoul/article/202411060956001
(... 생략 ...)
이제 모든 필요한 정보를 수집했으므로, 최종 기사를 작성하겠습니다.

### 제목: 서울월드컵경기장 잔디 문제, 예산 증액에도 불구하고 여전히 난항

#### 부제: 관리 부실과 정책적 대응 미비, 잔디 상태 악화 지속

#### 리드문:
서울월드컵경기장의 잔디 상태가 여전히 개선되지 않아 큰 논란을 일으키고 있다. 서울시는 잔디 관리 예산을 33억 원으로 증액했지만, 여전히 잔디 상태는 심각한 수준이다. 잔디 문제는 경기의 질을 저하시키고 국제 경기 유치에도 부정적 영향을 미칠 수 있다.

#### 본문:
서울월드컵경기장은 한국 축구의 상징적인 경기장으로서, 다양한 국제 경기를 개최하며 많은 축구 팬들의 주목을 받는 장소이다. 하지만 최근 잔디 상태 악화로 인한 경기 질 저하가 심각한 문제로 떠오르고 있다. 서울시는 잔디 관리 예산을 33억 원으로 3배 증액했으나, 그 효과는 미미하다는 평가가 나오고 있다.
(... 생략 ...)
결론적으로, 서울월드컵경기장의 잔디 문제는 단순히 관리적 차원을 넘어 정책적 대응이 필요한 복합적인 문제이다. 장기적인 해결책 마련이 시급하며, 서울월드컵경기장의 잔디 문제는 향후 국제 경기 유치와도 직결될 수 있는 중요한 문제로 인식되고 있다. 교훈을 얻어 근본적인 대책을 마련해야 할 시점이다.
```

구조가 단순하지만 랭그래프를 이용해 기사를 작성하는 챗봇을 만들어 보았습니다. 이어지는 실습에서 더 다양한 기능을 가진 멀티에이전트를 만들어 보겠습니다.